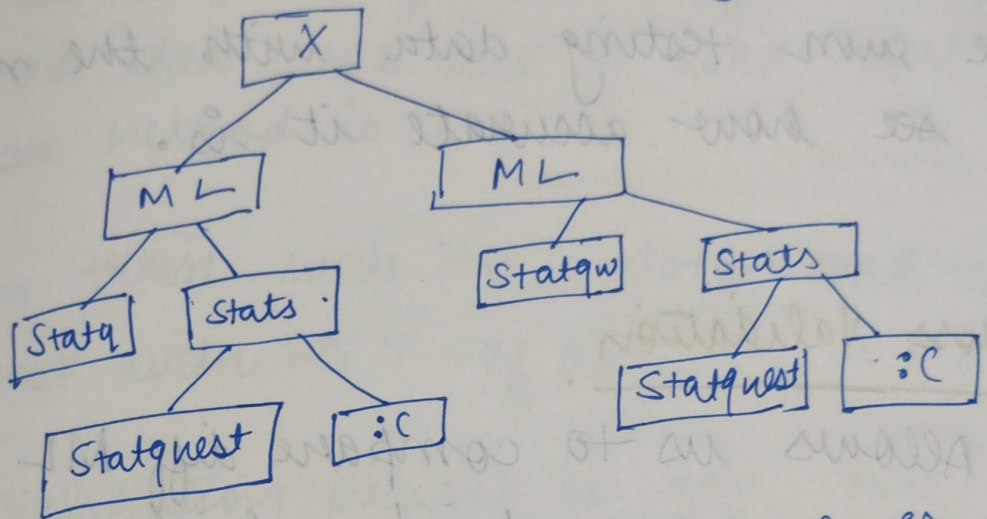


A Gentle intro to Machine Learning (Statquest ML)

Decision tree

↳ Predict behaviours of something.



Decision tree classifies if someone loves statquest or not.

→ Machine Learning is all about making predictions & classification.

→ Original data is called Training Data

→ Don't be fooled by how well a machine learning method fits the training Data.

→ Fitting the training Data well but making poor prediction is called bias variance.

Tradeoff.

* Most important thing • then any fancy model is how it goes with training data / Testing data.

→ we run testing data with the model to see how accurate it is.

Cross Validation

→ Allows us to compare diff ML methods and compare find which works best with training data.

2 Things to do with Data

- ① Estimate the parameter for ML methods.
- ② Evaluate how well the ML method works.

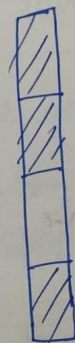
→ Estimating parameter is called Training the algorithm.

Basically we need to train & test the ML methods.

→ Reusing same Data for training & testing is a bad idea. We need to learn how well data method is.

→ We can train with 75% data and test with 25% of data.

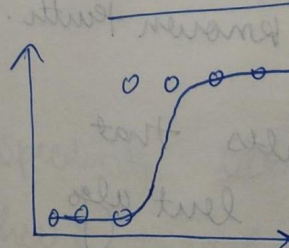
→ cross validation decides it for us



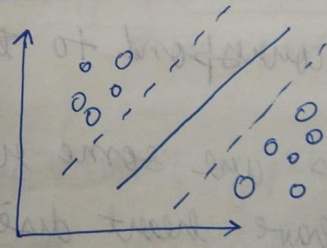
→ This dividing in 4 is called four fold cross validation using multiple

combo of 75% & 25%. we decide the best.

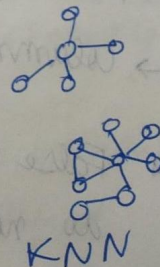
We then test which method performed the best



Logistic Regression



Support Vect machine



KNN

→ Tuning parameters

→ A parameter is not estimated but just guessed.

Confusion Matrix

We need to summarise how each method performs on the testing data.

We do this by creating a confusion matrix.

		Has heart disease	Does not have HD
Rows	Has Heart Disease	True Pos	False +ve
	Does not have heart disease	False Negative	True Negative

→ Rows predict what ML algo predicted

→ Columns correspond to the known truth.

* False +ves are some results that do not have heart disease but algo says that they do.

Now we make algo for each and then compare the confusion matrix.

→ Ultimately the size of confusion matrix is determined by number of things we want to predict.

Sensitivity & Specificity

In confusion matrix, rows correspond to what was predicted & columns the truth.

Once we fill confusion matrix we can find

→ Sensitivity
→ Specificity.

• Sensitivity tells what % was detected correctly.

$$\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Neg}}$$

• Specificity tells us what % of patients without heart disease were correctly identified

$$\text{Specificity} = \frac{\text{True Negative}}{\text{True Negative} + \text{False +ve}}$$

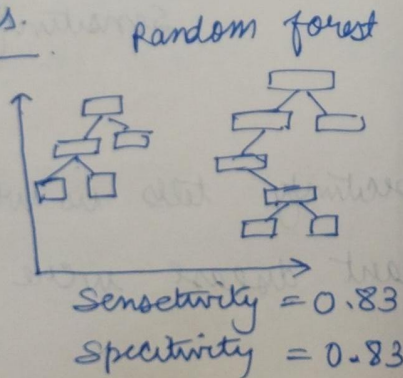
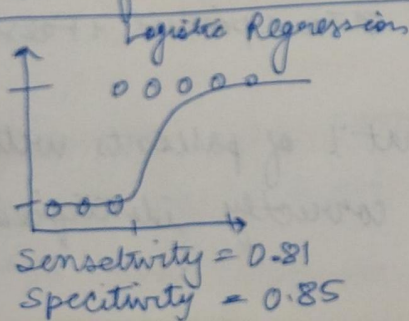
	Has Heart Disease	Does Not Have HD
Has HD	139 TP	20 FP
Does Not have HD	32 FS	112 TN

$$\text{Sensitivity} = \frac{139}{139 + 32} = 81\%$$

$$\text{Specificity} = \frac{112}{112 + 20} = 85\%$$

85% without heart disease were correctly identified.

Now for both models.



Sensitivity tells us that Random forest is slightly better at identifying +ves.

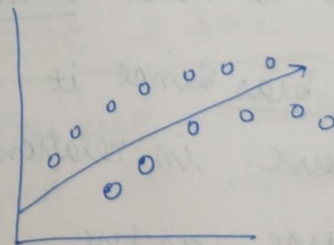
→ We would choose the logistic Regression model if correctly identifying patients ~~with~~ without heart disease was more important than with heart disease.

For 3X3 (or larger)

→ We calculate diff sensitivity ^{for} category

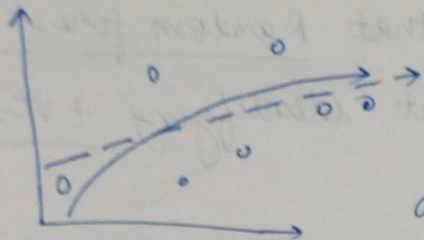
→ Formula stays the same.

Bias And Variance

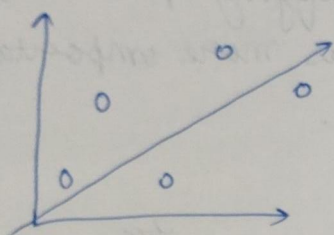


A straight line will not be able to fit into this curved shape.

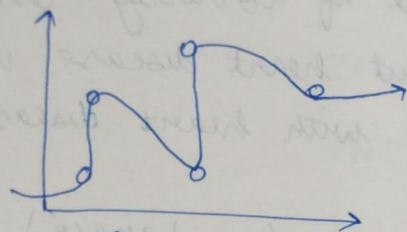
→ The inability for a ML method to capture true relationship is called bias.



Because the straight line can't be curved like a true relationship it has a large bias



The sum of sq of errors



This is better for the training set

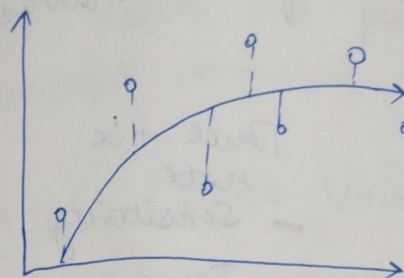
But if we fit the testing set now, the ~~straight~~ ^{curvy} line has $\gg$ error square

→ The diff b/w data sets is called variance

→ Straight line has high bias since it cannot capture the curve in relation

→ But has low variance as the sum of error distance is similar

when the squigly line fits the training data really well but not the testing set we say it as an overfit.



→ This one is perfect
→ Low variability
→ A sweet spot b/w simple model and complex model

→ 3 common methods to find sweet spot

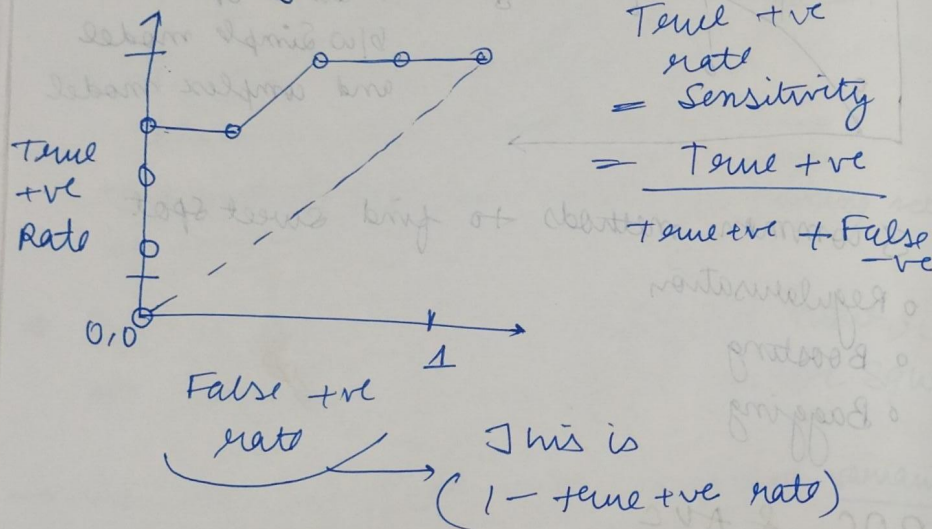
- Regularisation
- Boosting
- Bagging

ROC & AUC

We need a threshold to classify stuff and find the right amount of acceptable ~~the~~ false positives or true negatives for the dataset.

We need to determine the best threshold.

→ ROC Graph summarises all the info about most confusion matrices on the graph.



Actual

	True +ve	False +ve
Predicted	false -ve	True -ve

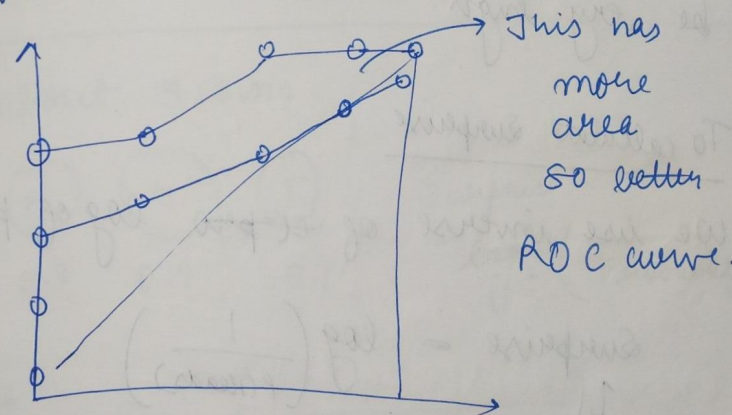
The more we move left in the ROC graph, the true +ve proportion is more than false +ve.

→ ROC solves the problem of creating multiple confusion matrix.

AUC (Area Under the Curve)

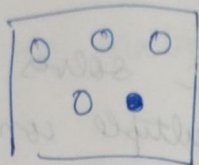
→ Tells us which ROC curve is better

→ More the area, better the ROC curve.



Entropy For Data Science

→ Entropy is useful for a lot of things in Data Science



SURPRISE

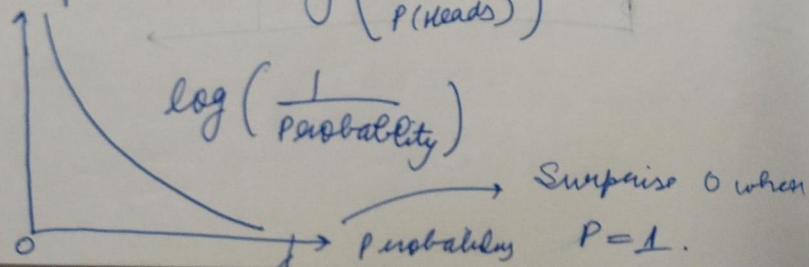
If there are 5 chickens in A box & 4 are red & 1 is blue.

If we pick up the blue box one randomly, the surprise factor will be very high

To calculate surprise

We use inverse of ~~a prob~~ log of probab.

$$\text{Surprise} = \log\left(\frac{1}{P(\text{Heads})}\right)$$



When the output space is 2 (like head - tails) then the base for log is 2

$$\text{Surprise} = \log_2\left(\frac{1}{\text{Probability}}\right)$$

Example.

Coin gets Head 90% of Time

Coin gets Tail 10% of the Time

$$\text{Head} \rightarrow \log_2\left(\frac{1}{P(\text{head})}\right) = \log_2\left(\frac{1}{0.9}\right) = 0.15$$

$$\text{Tail} \rightarrow \log_2\left(\frac{1}{P(\text{tail})}\right) = \log_2\left(\frac{1}{0.1}\right) = 3.32$$

What about 3 coins

$$\begin{array}{ccc} \text{H} & \text{H} & \text{T} \\ 0.9 & 0.9 & 0.1 \end{array}$$

$$\text{Surprise} = \log_2\left(\frac{1}{\text{Probability}}\right)$$

$$\Rightarrow \log_2\left(\frac{1}{0.9 \times 0.9 \times 0.1}\right)$$

$$\Rightarrow 3.62$$

	Head	Tails
$P(x)$	0.9	0.1
Surprise	0.15	3.32

what if we flip 100 times

$$\begin{aligned}
 &\text{Total Surprise for getting head} \\
 &\rightarrow (0.9) \times 100 \times 0.15 = \quad \quad \quad \text{Add} \\
 &\text{Total Surprise for Tails} \\
 &\rightarrow (0.1) \times 3.32 \times 100 = \quad \quad \quad = 46.7
 \end{aligned}$$

Avg Ammm of surprise per coin toss

$$\rightarrow \frac{46.7}{100} = 0.47$$

* Entropy $\rightarrow$ Expected surprise every time we flip the coin

$$= \sum x P(x=x)$$

Replacing the values.

$$\text{Entropy} = \sum \log\left(\frac{1}{P(x)}\right) \underbrace{P(x)}_{\substack{\text{Prob of} \\ \text{surprise}}}$$

$\downarrow$ Surprise

in books we see:

$$\text{Entropy} = - \sum P(x) \log(P(x))$$

Mutual Information

How much each one can tell us about the thing we want to predict.

$\rightarrow$ We can use R^2 to find mutual information, but R^2 works only with continuous data.

$$\text{Mutual Information} = \sum \sum P(x,y) \log \left[\frac{P(x,y)}{P(x)P(y)} \right]$$

$P(x,y) \rightarrow$ Joint probability

$P(x) \rightarrow$ Individual Proper Prob.

	Likes Popcorn Y	Likes Popcorn No	P(Totals 2) Row Totals
Loves Total Y	3/5	1/5	4/5
Loves Total No	0/5	1/5	1/5
P(Popcorn) Column Tot	3/5	2/5	

→ The two summations ensure that we include all possible combination of Var

$$MI = 3/5 \log \left[\frac{3/5}{3/5 \times 4/5} \right]$$

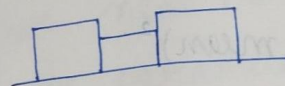
$$+ 0/5 \log \left[\frac{0/5}{3/5 \times 1/5} \right]$$

$$+ 1/5 \log \left[\frac{1/5}{2/5 \times 4/5} \right]$$

$$+ 1/5 \log \left[\frac{1/5}{2/5 \times 1/5} \right]$$

So mutual info is = 0.22.

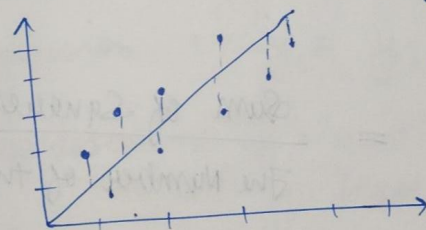
→ When two columns change, they change in the exact same ways, the mutual information is close to 0.5.



→ we create histogram of continuous values. and one or more is cont.

Linear Regression

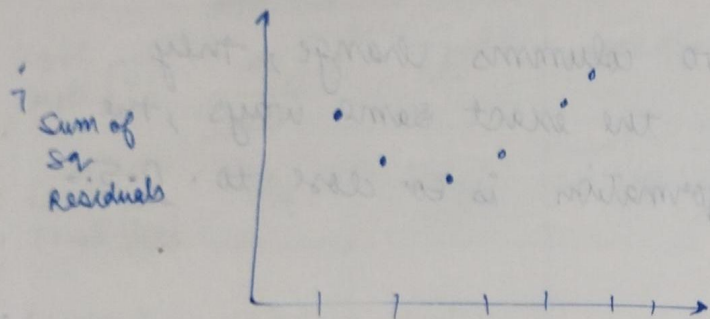
(1) Use least square to fit a line in the data.



- Calculate R^2
- Calculate P value of R^2

Distance from a line to a data point is called residual.

We can plot the sum of square residuals



$$SS(\text{mean}) = (\text{data} - \text{mean})^2$$

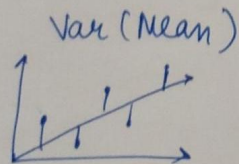
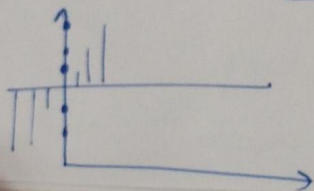
$$\frac{\text{Variation around the mean}}{\text{mean}} = \frac{(\text{data} - \text{mean})^2}{n}$$

sample size

→ In general

$$\text{Variance (something)} = \frac{\text{Sum of Squares}}{\text{The Number of those things}}$$

$$R^2 = \frac{\text{Var}(\text{mean}) - \text{Var}(\text{Fit})}{\text{Var}(\text{Mean})}$$



Steps

→ Generate set of random data.

→ Calculate mean & $SS(\text{mean})$

→ Calculate fit & $SS(\text{fit})$

$$F = \frac{SS(\text{mean}) - SS(\text{fit}) / P_{extra}}{SS(\text{fit}) / n - p_{fit}}$$

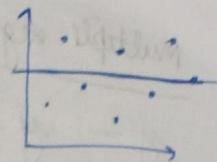
→ we need to minimise the number of parameters.

→ P value is smaller when there are more samples relative to the number of parameters in the fit.

Simple regression → $y = y\text{-intercept} + \text{slope } x$

Multiple regression → $y = y\text{-intercept} + \text{slope } x + \text{slope } z$

we calculate R^2 for both cases.



Multiple regression

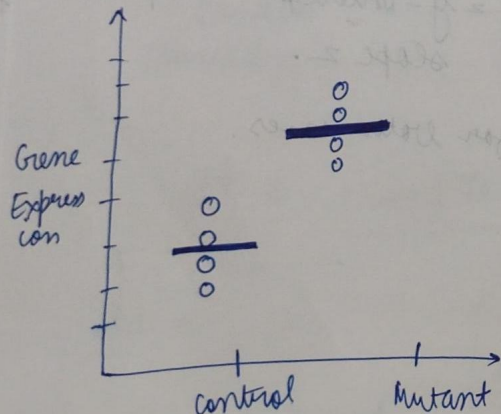
$$y = y\text{-intercept} + \text{slope}_x \cdot x + \text{slope}_z \cdot z$$

$$R^2 = \frac{SS(\text{Mean}) - SS(\text{fit})}{SS(\text{Mean})}$$

For multiple regression you adjust R^2 to compensate for additional parameters in the equation.

$$F = \frac{SS(\text{mean}) - SS(\text{fit}) / (p_{\text{fit}} - p_{\text{mean}})}{SS(\text{fit}) / (n - p_{\text{fit}})}$$

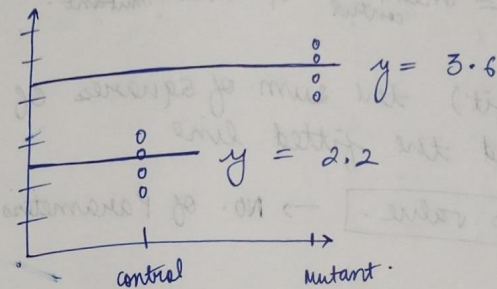
using Linear Model for t -test, ANOVA



T-test is used to check if means are significantly different from each other.

T-test (with linear Regression)

- ① Ignore the x-axis & find overall mean
- ② Calculate SS (mean) around the mean
- ③ Fit line to the data.
→ Do same for mutant data



$y = 1 \times 2.2 + 0 \times 3.6 + \text{residual}$
 $y = 1 \times 2.2 + 0 \times 3.6 + \text{residual}$
 $y = 1 \times 2.2 + 0 \times 3.6 + \text{residual}$
 $y = 1 \times 2.2 + 0 \times 3.6 + \text{residual}$
 $y = 0 \times 0.22 + 1 \times 0.36 + \text{residual}$
 $y = 0 \times 0.22 + 1 \times 0.36 + \text{residual}$
 $y = 0 \times 0.22 + 1 \times 0.36 + \text{residual}$
 $y = 0 \times 0.22 + 1 \times 0.36 + \text{residual}$

→ This gives design for the

→ This gives us a design matrix for the same.

L
T₀

L
-

f

1	0
1	0
1	0
1	0
0	1
0	1
0	1
0	1

The design matrix can be combined with an abstract version of equation to fit in the data.

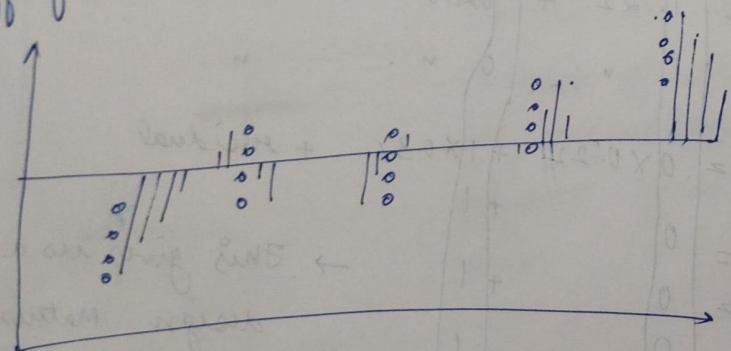
$$y = \text{column } 1 \times 2.2 + \text{column } 2 \times 3.6$$

$$y = \text{mean}_{\text{control}} + \text{mean}_{\text{mutant}}$$

- ② Calculate $SS(\text{fit})$ the sum of squares of residuals around the fitted line & find the P value. $\rightarrow$ No. of Parameters

ANOVA

Checks if mean of two or more groups is diff from each other.



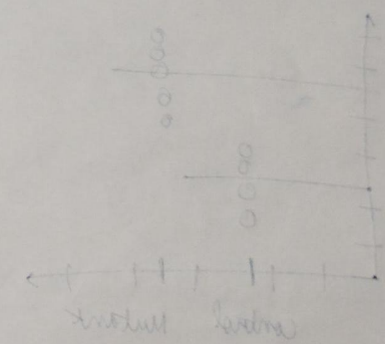
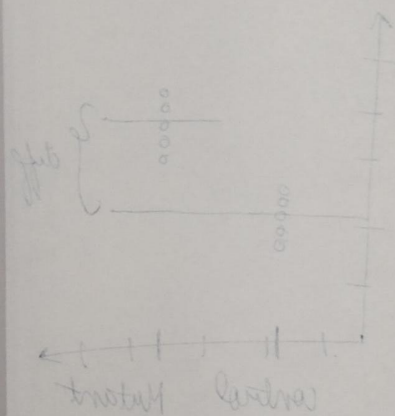
$$y = \text{mean expression} = p \cdot \text{mean} = 1$$

$$y = \text{mean}_1 + \text{mean}_2 + \text{mean}_3 + \text{mean}_4 + \dots + \text{mean}_m$$

here $p_{\text{fit}} = 5$ ($m = 5$)

$$\frac{(x_j - \bar{x})^2}{(x_j - \bar{x})^2} = 1$$

$$\frac{(x_j - \bar{x})}{(x_j - \bar{x})} = 1$$



Design Matrices for Linear models.

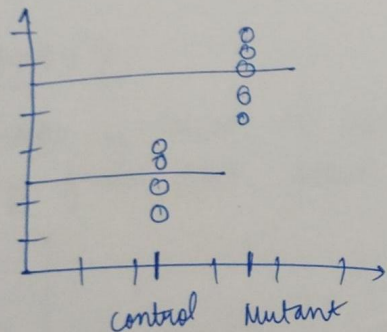
$$y = \text{mean (control)} + \text{mean (mutant)}$$

→ same no. of parameters.

→ p value comes out to be same

$$P_{fit} = 2$$

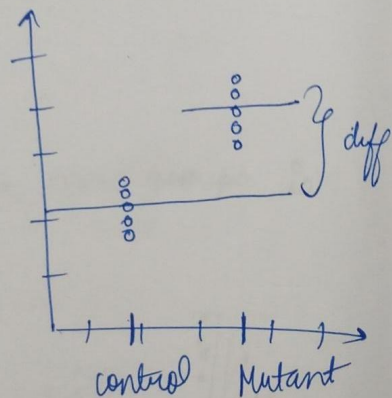
→ Same F = Same P value



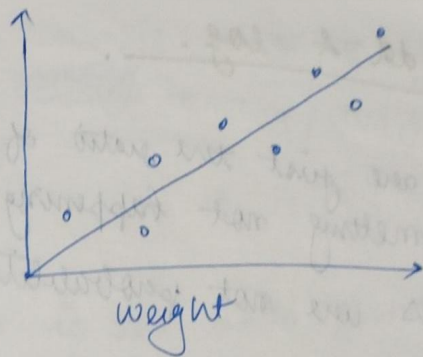
$$y = \text{mean (control)} + \text{diff (mutant-control)}$$

→ same parameter = 2.

$$F = \frac{SS(\text{mean}) - SS(\text{fit})}{P_{fit} - P_{\text{mean}}} \div \frac{SS(\text{fit})}{(n - P_{fit})}$$



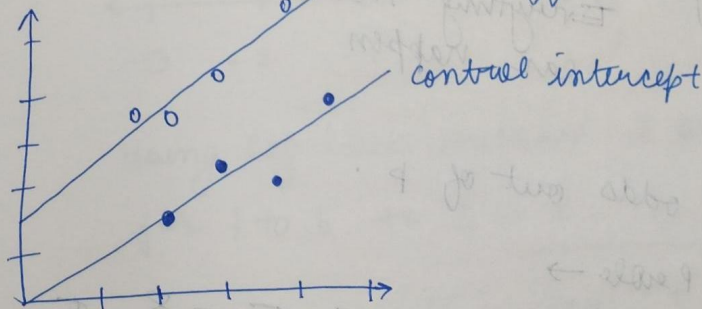
1	0.9
1	1.6
1	2.3
1	3.5
1	4.2
1	5.1
1	5.5
1	5.6



$$y = 1 \times y\text{-intercept} + 1.6 \text{ slope}$$

These are the points we plug in the equation to fit in the line

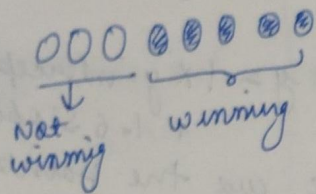
combining t-test & regression mutant offset.



$$y = \text{control intercept} + \text{mutant offset} + \text{slope}$$

Odd & log.

- Odds are just the ratio of something happening to something not happening
- Odds are not probabilities



$$\text{Odds} = \frac{\text{Team winning}}{\text{Team not winning}} = \frac{5}{3} = 1.7$$

$$\text{Probability} = \frac{\text{My Team winning}}{\text{Everything that can happen}} = \frac{5}{8} = 0.625$$

* Finding odds out of P.

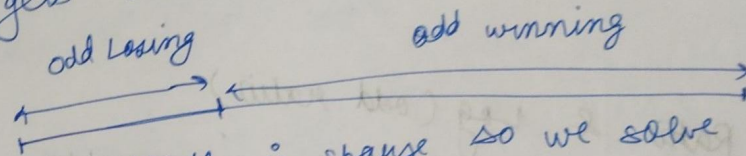
Losing Probable →

$$1 - \text{prob of winning} = 1 - \frac{5}{8} = \frac{8}{8} - \frac{5}{8} = \frac{3}{8} = 0.375$$

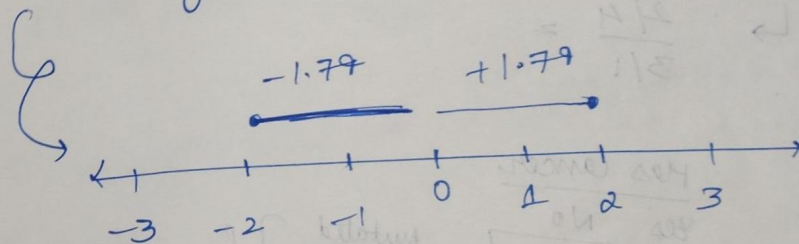
Now →

$$\frac{\text{ratio of prob of winning}}{\text{prob of losing}} = \frac{5/8}{3/8} = \frac{5}{3} = 1.7$$

→ The worse team is the odds of winning gets closer & closer to 0.



magnitude is sparse so we solve this by taking the log.

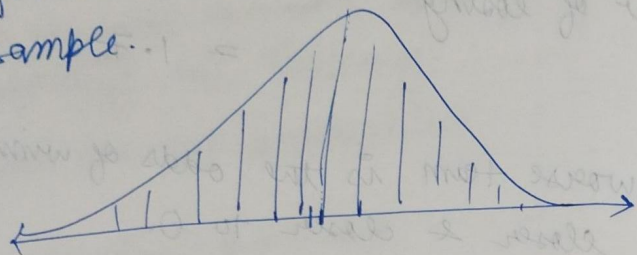


using log the distance is same for 1 to 6 to 6 to 1.

$$\log(\text{odds}) = \log\left(\frac{5}{3}\right) = \log\left(\frac{P}{1-P}\right)$$

This is something called logistic regression.

→ We do this because Log of the odds follow normal distribution of a random sample.



Odd Ratio & Log (odd ratio)

→ odds ratio means the ratio of the odds.

$$\rightarrow \frac{2/4}{3/1} =$$

		Has Cancer	
		yes	No
Has mutated gene.	yes	23	117
	No	6	210

mutated gene with cancer

$$\rightarrow \frac{23}{117} = \frac{6}{210}$$

$$\Rightarrow \frac{0.2}{0.03} = 6.88$$

$$\log(6.88) = 1.93$$

Larger value means good predictor of cancer.

→ ways to determine if odd ratio is statistically significant →

(1) Fishers Exact Test.

(2) Chi Square Test.

(3) The Wald Test.

• Chi Square Test

$$P(\text{Has cancer}) = \frac{24}{356} = 0.08$$

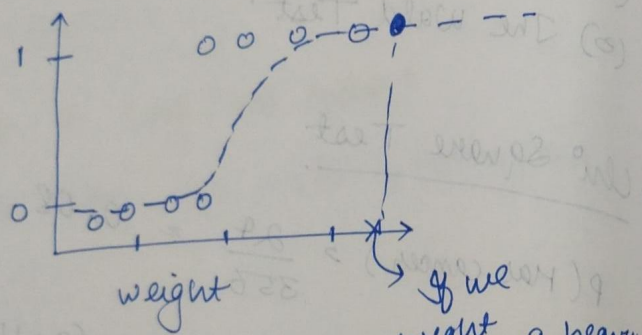
Expected value. → $0.08 \times (23 + 117) = 11.2$

11.2	128.8
17.3	198.7

Logistic Regression

→ Logistic Regression Predicts something is true or false instead of something like size.

→ Fits an S shape into a function



→ Logistic Reg tells us about classification

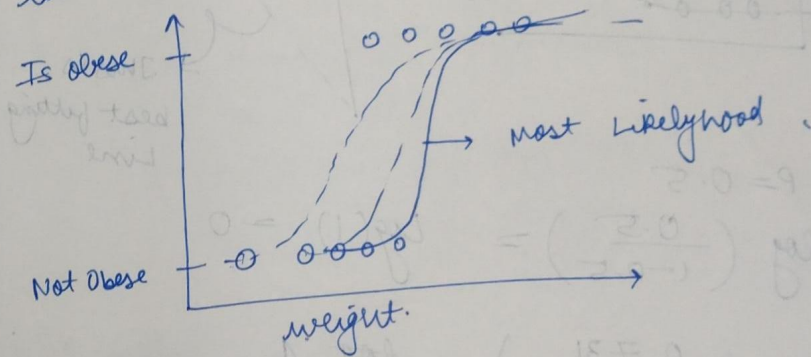
→ We can make simple models
★ Obesity predicted by weight

→ Or complicated models.

★ Obesity is predicted by weight + Genotype

→ Logistic Regression works with continuous data. (Like weight & age)
or
Discrete data (genotype & astrosign)

→ Curve with maximum likelihood of the data is selected.



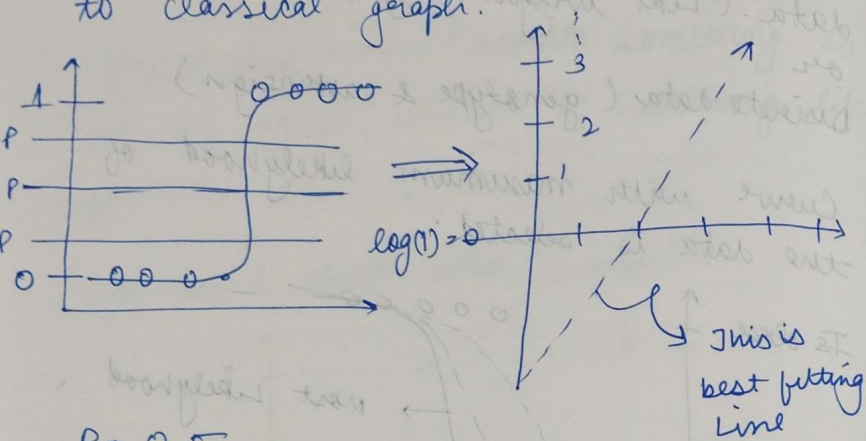
Logistic Regression - 1 - coefficients

→ Logistic Regression is a Generalised Linear Model

→ Y axis values in Logistic regression is confined b/w 0 & 1.

$$\log(\text{odds of obesity}) = \log\left(\frac{p}{1-p}\right)$$

converting the logistic Regression graph to classical graph.



$$P = 0.5$$

$$\log\left(\frac{0.5}{1-0.5}\right) = \log(1) = 0$$

$$\log\left(\frac{0.73}{1-0.73}\right) = \log 1$$

$$\log\left(\frac{0.88}{1-0.88}\right) =$$

→ Coeff using Linear Logistic Reg

Estimated Std

Error

z value

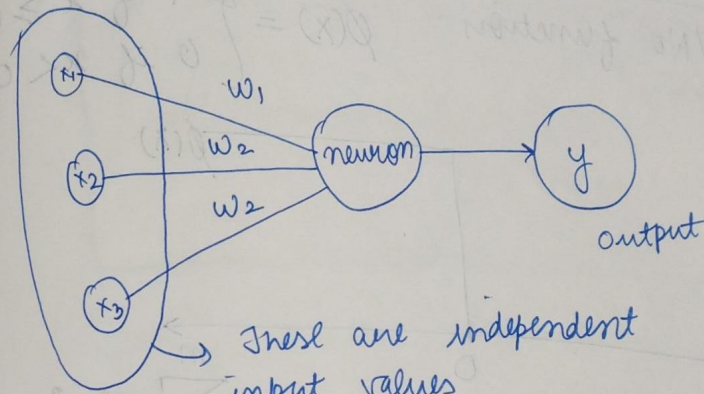
-3.476

2.364

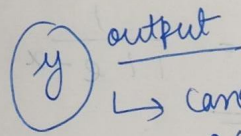
-1.471

$$y = -3.47 + 1.83 \times \text{weight}$$

Deep Learning (Rishik Bhaiya)

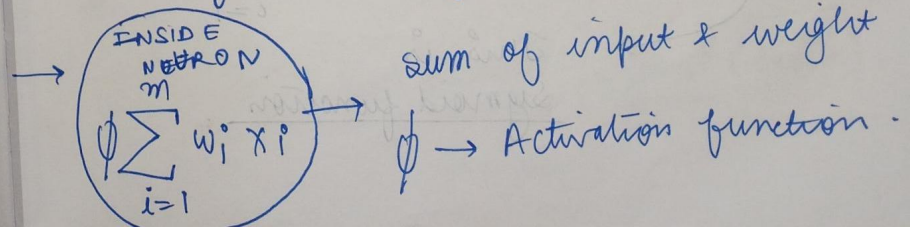


These are independent input values that are standardised to a certain range.



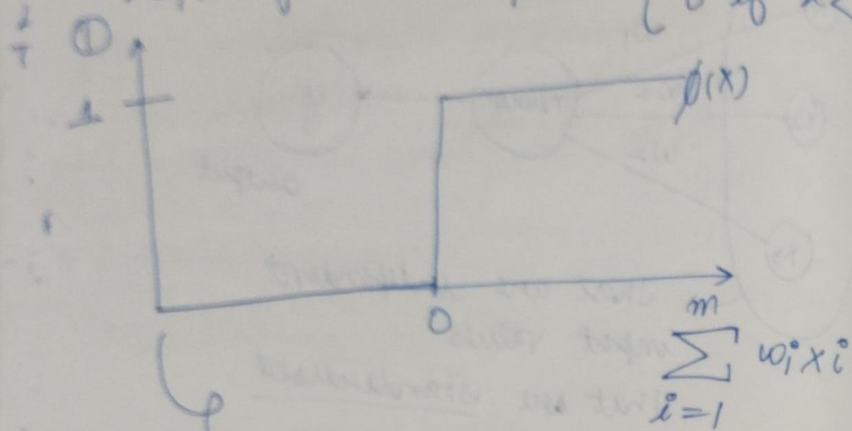
- continuous (price)
- Binary (will exit yes/no)
- Categorical.

→ $w_1 - w_3$ are weights are ~~terms~~ adjusted according to the gradient descent.

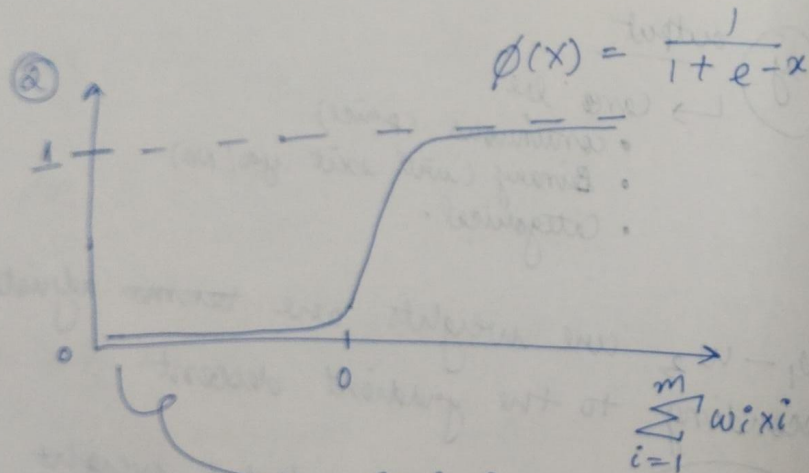


The Activation Function

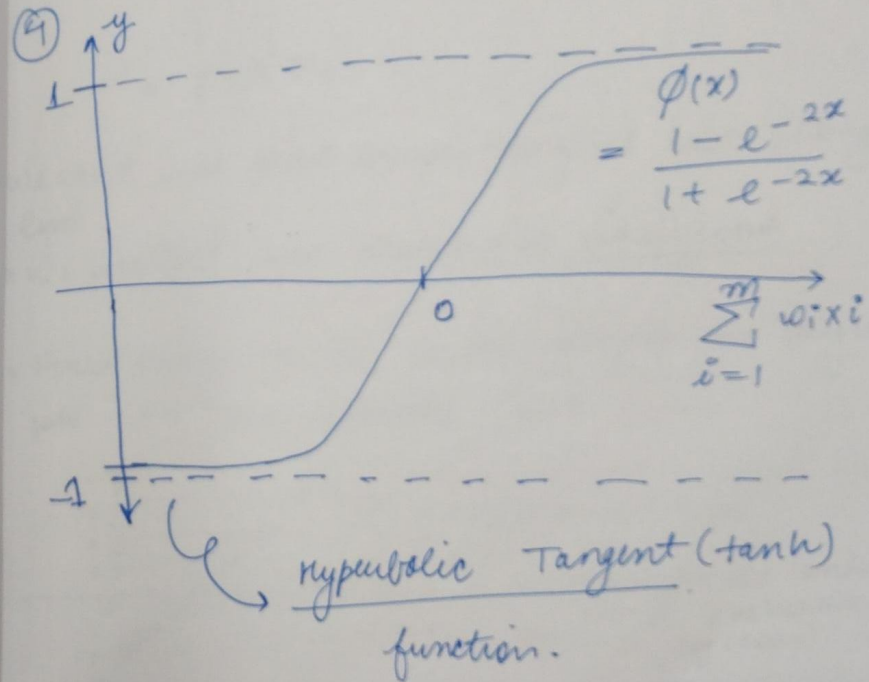
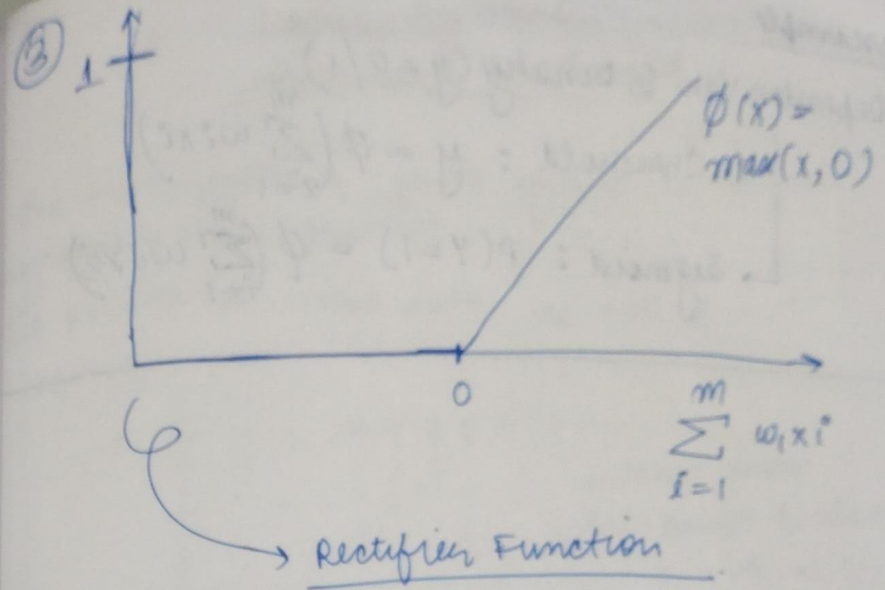
→ Yes/No function. $\phi(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$



→ This is threshold function



→ This is Sigmoid function



example

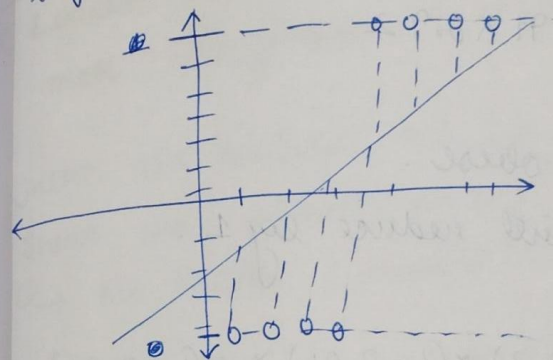
Dependent Var is binary ($y = 0/1$)

→ Threshold: $y = \phi\left(\sum_{i=1}^m w_i x_i\right)$

→ Sigmoid: $P(Y=1) = \phi\left(\sum_{i=1}^m w_i x_i\right)$

Logistic Regression maximum Likelihood

→ The transformation from logistic to normal regression has a problem
It pushes the raw data to +ve & -ve infinity.

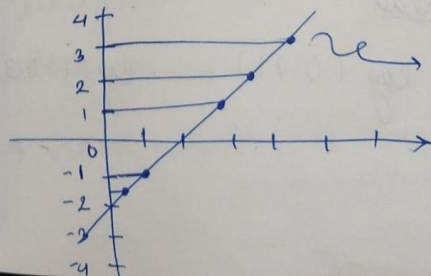


distance from data points is also
-ve & +ve ∞

→ We can't use least square to find best fitting line

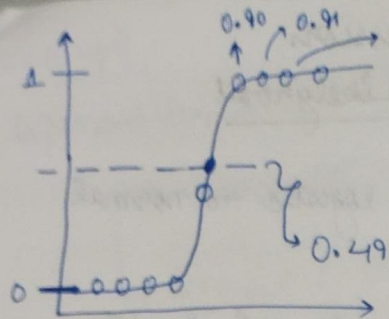
→ We instead use maximum likelihood.

→ First thing to do is to project the original pts. to the candidate line



→ projecting the lines

• we transform the candidate
log odds to probabilities
→ $P = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$



Probability is not area under the curve but instead y-axis value.

Likelihood for obese mouse.

$$\rightarrow 0.49 \times 0.9 \times 0.91 \times 0.92$$

Likelihood for not obese.

$\rightarrow$ Not obese will reduce by 1.

$$\rightarrow (1 - 0.9)$$

$$\rightarrow x(1 - 0.9) \times (1 - 0.3) \times (1 - 0.01) \times (1 - 0.01)$$

Total likelihood

$$\rightarrow 0.49 \times 0.9 \times 0.91 \times 0.92 \times (1 - 0.9) \times (1 - 0.01)$$

We calculate the log instead

$$\rightarrow \log(0.49) + \log(0.9) + \log(0.91) + \dots + \log(1 - 0.3)$$

$$\log(\text{likelihood of given squiggle}) = -3.73$$

we calculate the likelihood for each notation.
 $\rightarrow$ Transforming log odds to probabilities and finding smaller log (likelihood)

Logistic Regression - Fitting a line with max likelihood.

There are multiple R^2 in logistic Regression

There are multiple R^2 but we are using Mc Fadden Pseudo R^2

$R^2 \rightarrow$ Normal $\rightarrow$ Percentage of variation around mean when fitting line to data.

$$R^2 = \frac{SS(\text{Mean}) - SS(\text{Fit})}{SS(\text{Mean})}$$

$\rightarrow$ When the model is a good fit the log likelihood for logistic regression is a value close to zero.

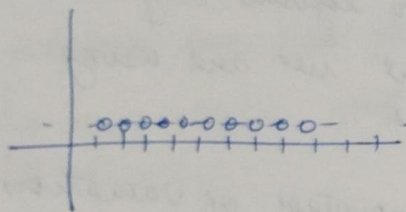
$\rightarrow$ Good fits have LL closer to 0

Calculating p-value

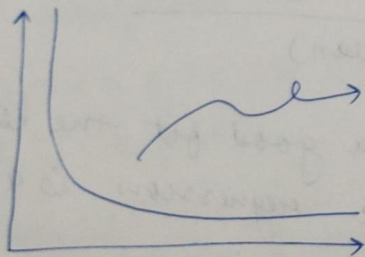
$$2(LL(\text{fit}) - LL(\text{Overall P}))$$

= Chi Square value with degree of Freedom equal to number of parameters in 2 models.

$$\text{DOF} = 2 - 1 = 1$$



LL(Overall P) has 1 parameter since it only needs an estimator for y-intercept.



Chi Square with DOF = 1.

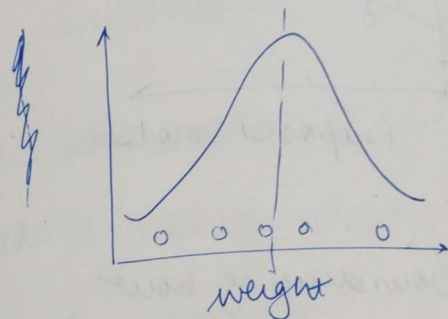
LL(fit) is generally closer to 0 than LL probability and since LL are -ve
 $2(LL(\text{fit}) - LL(OP)) = \text{Chi Sq value}$

LL of saturated model is generally 0.

$$R^2 = \frac{LL(\text{Overall P}) - LL(\text{Fit})}{LL(\text{Overall P}) - \underbrace{LL(\text{Saturated Saturated})}_0}$$

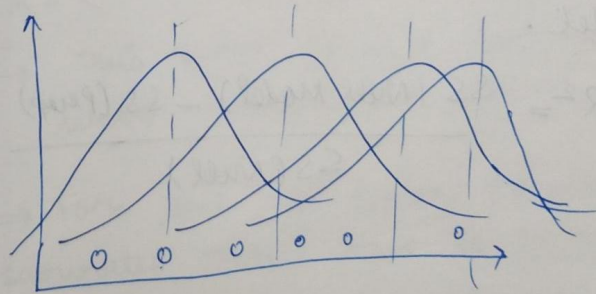
Saturated models and deviance

We have a sample size and we draw normal distribution of the curve.



Since we only need to estimate the mean for this model, we can say we have 1 parameter.

1 parameter model is called null model.

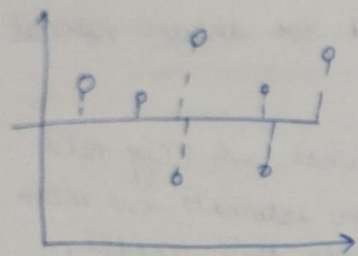


There is 1 parameter for one data point. This is called saturated model.

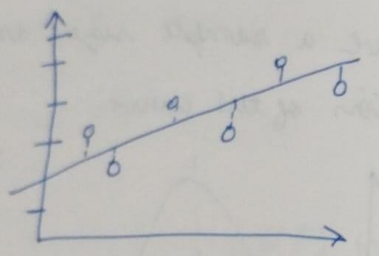
Likelihood of this data = $3.3 \times 3.3 \times 3.3 \times 3.3 = 1241.5$

- Null model gives out worst case scenario
- Saturated model gives out the best case scenario.

We use Null & Saturated models to calculate R^2 & p-value for proposed model.



Null model



Proposed model

Normal model has boundary of how poorly it performs, $SS(\text{Null model})$ & fixed value 0 that represents best possible model.

$$R^2 = \frac{SS(\text{Null Model}) - SS(\text{Prop})}{SS(\text{Null})}$$

However unlike traditional R^2 there is no fixed way / ideal fit for every situation, that is why we use LL (Saturated)

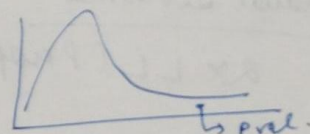
$$R^2 = \frac{LL(\text{Null Model}) - LL(\text{Proposed})}{LL(\text{Null Model}) - LL(\text{Saturated})}$$

If proposed model = Saturated model
then

$$LL(\text{Prop}) = LL(\text{Saturated})$$

$LL(\text{Saturated model})$ [Log Likelihood for SM]

adds a scaling factor



* Residual Deviance

$$2 \times (LL(\text{Saturated Model}) - LL(\text{Proposed}))$$

→ This when plugged into chi² value graph with 4 parameters gives us the p value.

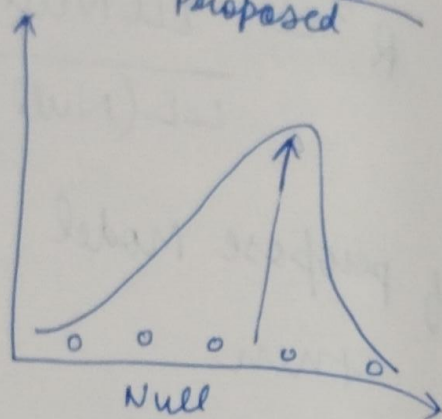
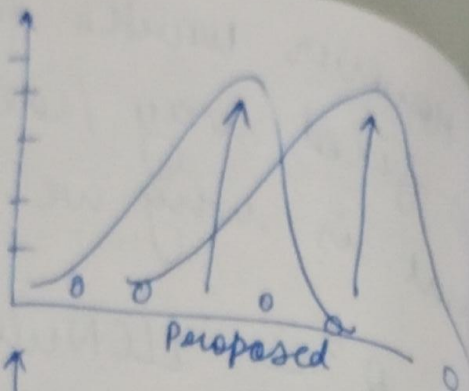
⇒ For this chi² Square to work, proposed and saturated models have to be nested.

In other words Proposed model is simpler version of saturated model.

- We can calculate the
p-value from log likelihood
 R^2 from difference in
deviance

Null Dev — Residual Deviance

↓
this is a chi-square
value with degree of
freedom equal to num
of parameters for prop
model & null model.



Residual Deviance

↳ $2 \times LL(\text{Proposed Model})$

Null Deviance

↳ $2 \times LL(\text{Null Model})$